

E-Cars

Design Document

Document Change Control

Version	Date	Authors	Summary of Changes
1	October 20/2023	EM,AA,WS,MS	Deliverable 1
2	November 18,2023	EM,AA,WS,MS	Deliverable 2
3	December 6, 2023	EM,AA,WS,MS	Deliverable 3

Document Sign-Off

Name (Position)	Signature	Date
Ali Adam	A.A	December 6
Eyad Mahmoud	E.M	December 6
Waleed Samouh	W.S	December 6
Malik Samouh	M.S	December 6

1 Introduction

1.1 Purpose

This document details the requirements for the system E-Cars. The goal here is to make a website that offers only electric cars for sale, giving the users the option to be admins and edit inventory or be a client that wants to buy a car simply. Offering a wide array of electric cars to choose from with features such as sorting from highest price to lowest as well as filtering by type and much more.

1.2 Overview

In this document, we give a detailed explanation of how we created our system to be able to handle all of these car sales done on the E-cars website. We used the Microservice architectural style to be able to execute the needed requirements. The client would invoke the UI and make requests which would be then sent to the gateway, the gateway. The gateway is an application of the front controller JEE pattern. This pattern was used to distribute the system on 4 different microservices. This ensures modularity, and fault isolation, and facilitates independent development and deployment. This is more cloud-native than using MVC because it has higher scalability which allows it to scale to larger sizes and be deployed to the cloud. The BFF (Backend for front end) design pattern was also used for the front end which was coded in react.

1.3 Resources - References

Include references to other documents that may assist in the understanding of this document. (i.e. past SRS and SDDs)

https://cs.uwaterloo.ca/~a78khan/courses-offered/cs446/2010_05/lectures/ides/12_je_patterns.pdf

<https://wisdomplexus.com/blogs/mvc-vs-microservices/#:~:text=with%20throughput%20time,-.How%20The%20Two%20Differ%3F,with%20each%20other%20using%20APIs.>

<https://developer.ibm.com/articles/the-sequence-diagram/>

<https://www.lucidchart.com/pages/uml-component-diagram>

<https://tiptopsecurity.com/how-does-https-work-rsa-encryption-explained/>

<https://docs.aws.amazon.com/ec2/>

<https://redux.js.org/>

<https://www.prisma.io/docs>

<https://nextjs.org/docs>

2 Major Design Decisions

Architecture:

The microservice architecture is used in this project, consisting of the following:

- Authentication Microservice:
 - Registering users and storing their information in the database
 - Hashing the password
 - Generating JWT tokens
 - Signing and verifying tokens
 - Logging in
- Catalog Microservice:
 - Add cars
 - Edit cards
 - View entire catalogue
 - View cars based on filters: Make, Model, year, type, history
 - Get Recommended cars
 - Get reviews for a car
 - Post a review for a car
- Shopping Cart Microservice:
 - Add items to cars
 - Remove items from cart
 - Get user's cart and
- Orders Microservice
 - Place an order for a user
 - Check catalogue to ensure quantity
 - Get users shopping cart items
 - Update catalogue quantity
 - Empty the users shopping cart
 - Record processes and denied orders
 - Handle payments
- Analytics Microservice
 - Record events for the car such as :View, Cart and Order
 - Generate usage report for the website by month
 - Generate sales report for a car by month

Dependency Injection:

Using an ORM to interact with the Database comes with support for dependency injection. An ORM will have the same APIs to interact with the MySQL database, Postgres or SQLite. The DB driver name can be provided to the ORM client while keeping the rest of the code the same

Having this dependency injection, the system can swap out between databases without having to change any of the backend design or implementation. Making it have low coupling.

DAO:

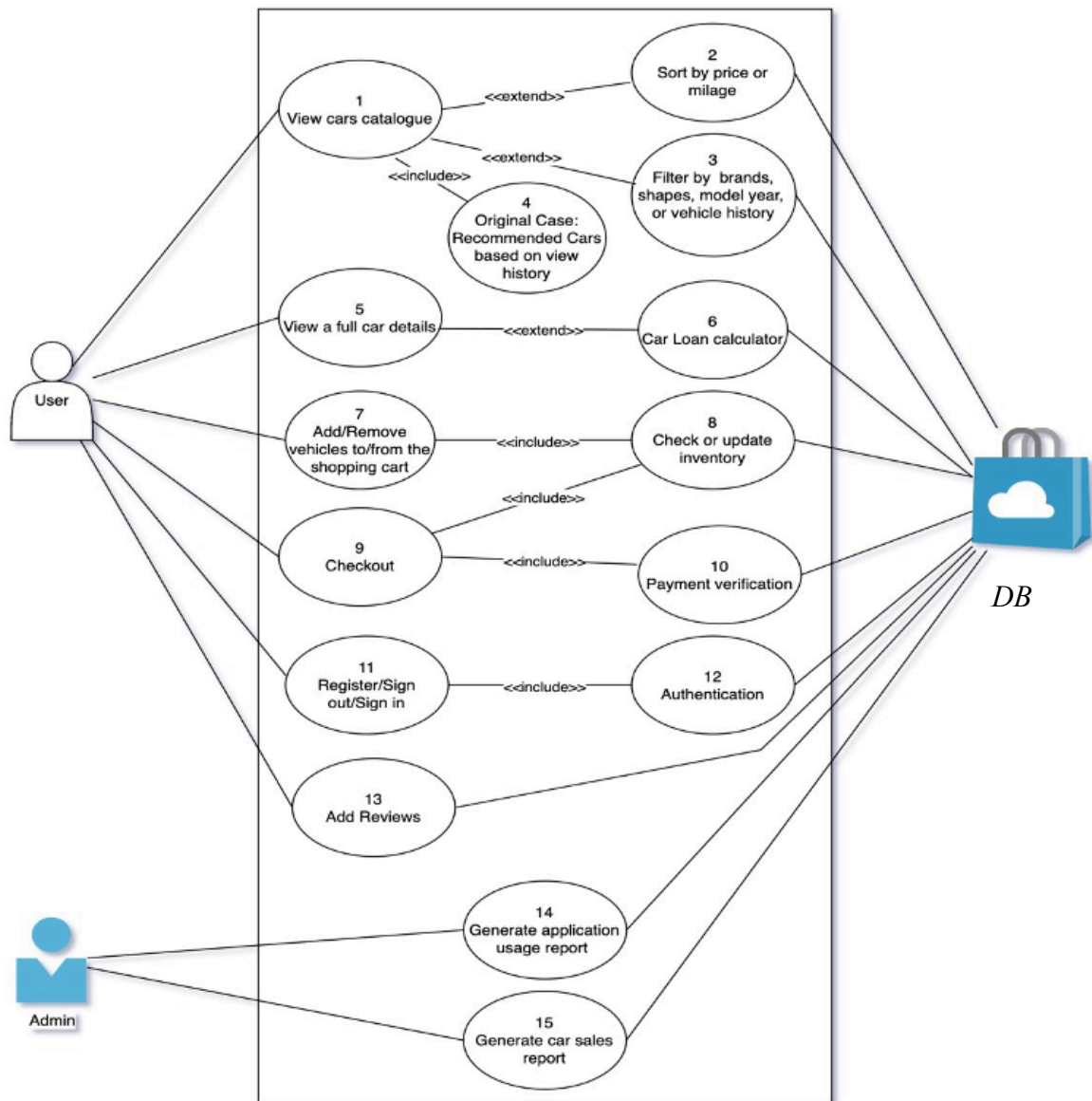
The Data access object will follow the singleton pattern, where there is only a single instance of this object, and therefore a single object that interacts with the database, this way, the controller and the services layer will be logically separated from the database, and there will be a single connection between the database and the rest of the backend. This method provides separation of logic between the business layer and the persistence layer

Middleware:

The middle pattern is used to intercept all requests coming to the controller, it acts as a layer prior to handling the request. Any request that fails the authentication or the authorization based on the JWT signature will be discarded.

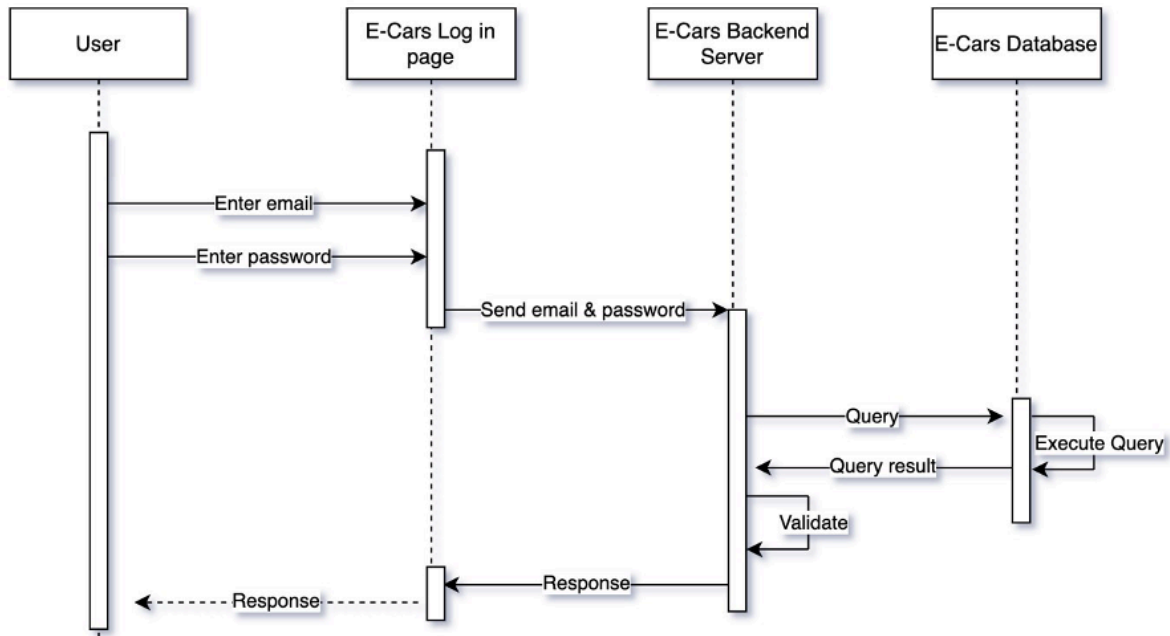
3 Use case Diagram

Provide a use case diagram depicting the features your e-commerce system will support.

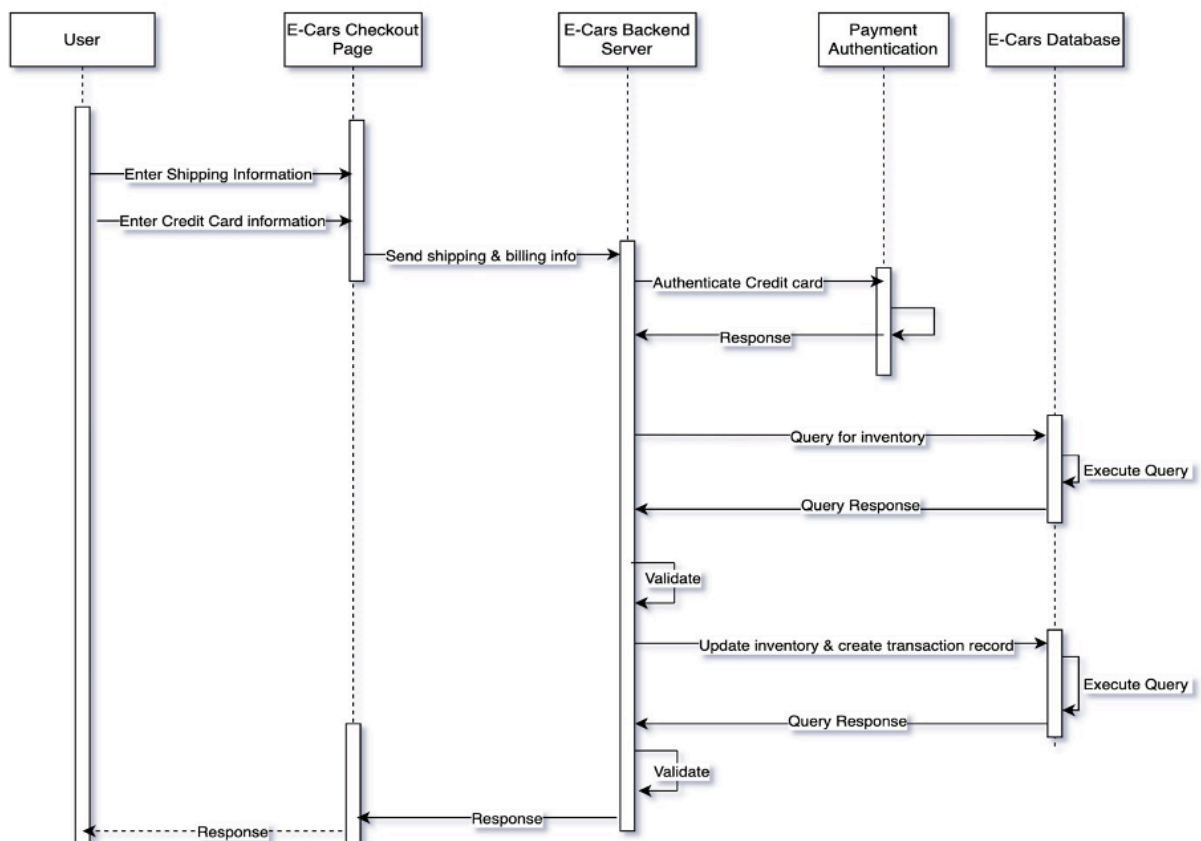


4 Sequence Diagrams

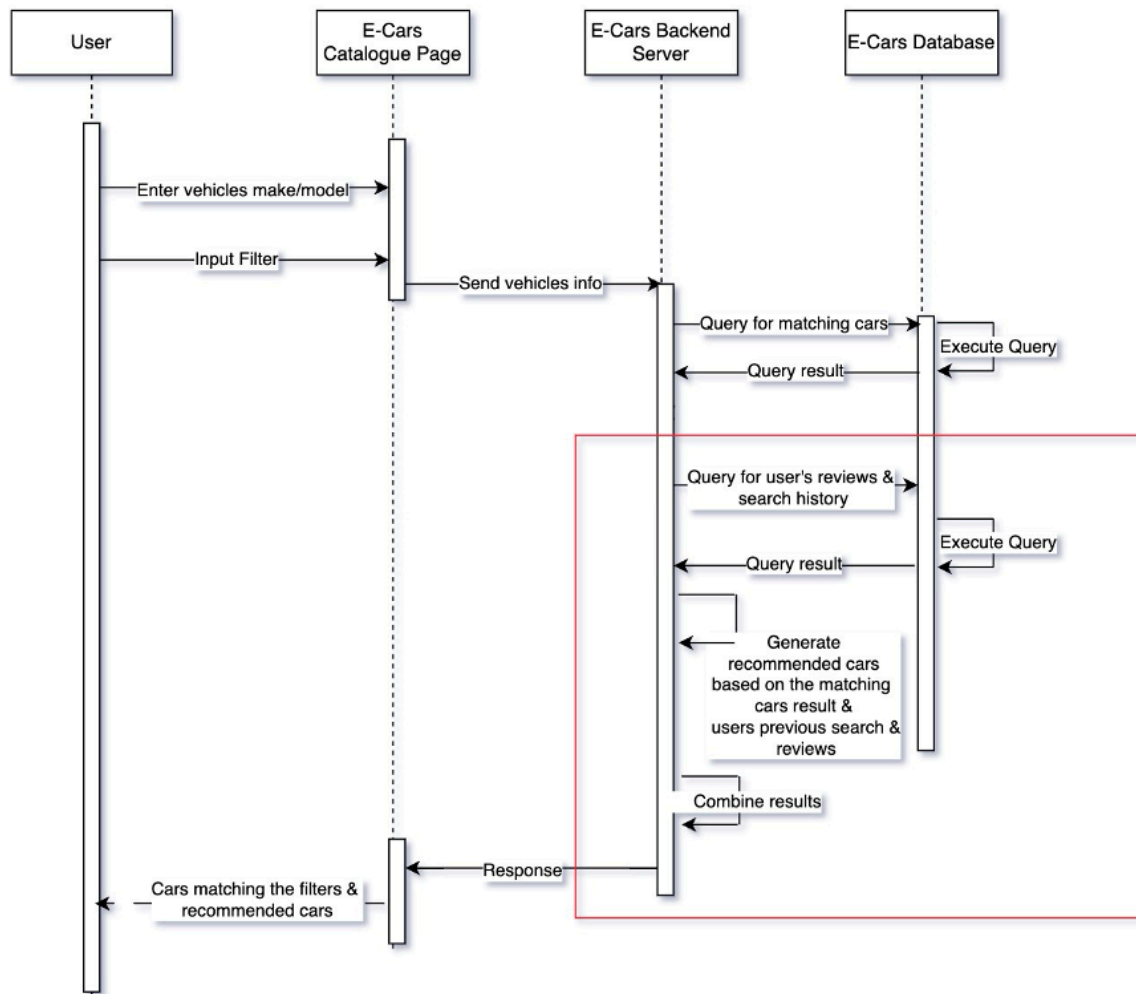
Use Case ID: 11



Use Case ID: 9

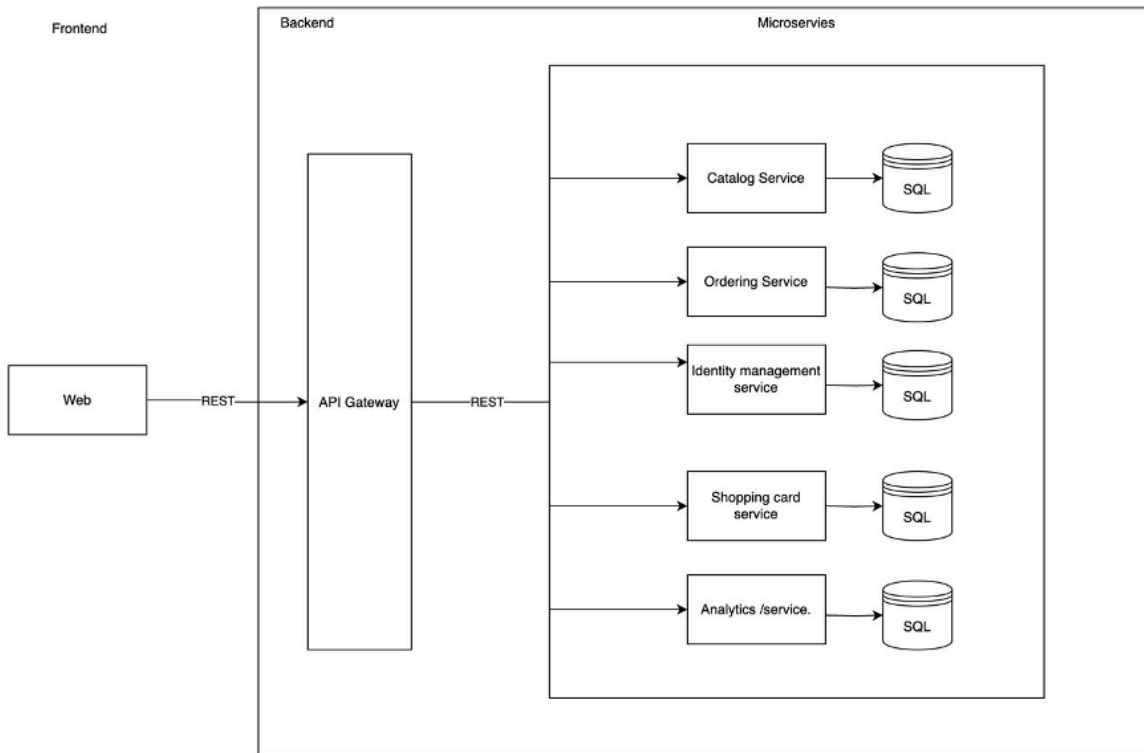


Use Case ID: 4 (In red)



5 Architecture

5.1 Microservices Architecture



5.2 UML diagrams

Modules			
Name	Function	Interface	Description
Services	to view cars in a catalog and allows sorting	CatalogServices Services: CatalogService	Interface that exposes Car services Interface that exposes Feature
Ordering Service	Interface that allows users to add items to cart and take out items from cart to place orders and make a purchase	OrderService	Interface that exposes services to place orders
Identity Management Service	Interface that deals with user authentication and authorization	AuthenticationService	Interface that exposes service for users to authenticate

g Card Service	that deals with ards to checkout	itCardService	terface that exposes rd to users
s / Servi	m that allows the o check analytics	alyticsService	nterface that exposes ytics system to the

Interface Names	Operations	Operation Descriptions
M1:I1	List<CarModel> I1: listAvaliableCars() : used by CarCatalogManager <CarModel> getCarDetails(carId : int) I1: used by CarCatalogManager List<CarModel> searchCars(KeyWord : String) I1 : Used by CarCatalogManager <PriceEstimate>: getCarPriceEstimate (CarId : int Location : String) I1 : Used by CarCatalogManager <ComparisonResult> CompareCars (carId : List <int>) I1 : Used by CarCatalogManager	Shows all available cars. Shows car details. Shows cars that correspond with our search. Gives the price estimate considering the area. Compares car details to each other.
M2:I2	List<Feature> : listFeaturesForCar (CarId) I2: Used by FeatureCatalogManager	Lists all featured cars

	<Feature> : getFeatureDetails(Feat ureId : int) I2: Used by M4 <CustomizedCar> : customizedCar(carId : int selectedFeatures : List<Features>) I2 : Used by M4	Gets all details from featured cars based on car chosen Selects customized cars based on the features selected
--	--	---

Interface	Operations	Operation Descriptions
M3: I1	<bool> : AuthenticateUser (User : user email: String Password : String) I1: Used by AuthenticateService <Token> generateToken(Us erID String) I1: Used AuthenticateServic e. <bool> validateToken(Tok enId String) I1 : Used AuthenticateServic e.	Authenticates if the email and password the user entered are correct. Generates token to authenticate users. Validates token for user to confirm identity

Component diagram



6 Activities Plan

6.1 Project Backlog and Sprint Backlog

<https://github.com/Ali-Adam7/E-Cars/issues?q=is%3Aissue+is%3Aclosed>

6.2 Group Meeting Logs (Deliverable 1)

Present Group Members	Meeting Date	Issues Discussed / Resolved
All	October 16	Major design decisions and architecture
All	October 20	Use Case, Sequence, Architecture, TDD

7 Test Driven Development

Test ID	1
Category	Registration and Log in
Requirements Coverage	UC-R1: Login
Initial Condition	System initialized, up and running
Procedure	1. User is on the login page. 2. User provides email and password in the form. 3. User clicks on the login button.
Expected Outcome	The inputs are sent to the backend, and checked against the DB to authenticate the user, if a matching email and password are found then the login is successful, otherwise if no match is found then the user is presented with an error message
Notes	The email will be checked if it's a valid email input before processing the request

Test ID	2
Category	Registration and Log in
Requirements Coverage	UC-R1: Logout
Initial Condition	The user is currently logged in
Procedure	1. User is on the account settings page. 2. User clicks on sign out.
Expected Outcome	The user state is cleared and will get logged out, redirecting the user to the homepage. From here the user will not have access to any features that require an account.

Test ID	3
Category	Registration and Log in
Requirements Coverage	UC-R1: Register
Initial Condition	System initialized, up and running
Procedure	1. User is on the registration page. 2. User provides email and password in the form, along with other information such as shipping address, phone number. 3. User clicks on the register button.
Expected Outcome	The inputs are sent to the backend and inserted into the database, then the backend sends a response to the frontend and the user is redirected to the homepage and is now authenticated. If another user has the same email then an error message will be displayed
Notes	If email verification is going to be used, then the backend will handle that and send the email to the user, awaiting him to click on a link to approve his email

Test ID	4
Category	Catalog
Requirements Coverage	UC-M1: browse the catalog and see All, By Brand, By Categories products
Initial Condition	System initialized, up and running
Procedure	1. User is on the home page 2. User can view catalog items or use an input form to filter by car make, car model, categories 3. User can sort by price, year, ascending or descending
Expected Outcome	All cars shown in the database shown to the user if no filters are selected. If user chooses to filter, then all cars shown will
	be of the same brand, category and/or year of the filter. And sorted asc/desc if that option was picked

Test ID	5
Category	Catalog
Requirements Coverage	UC-M2: select an item and see the detailed information for that item (price, ratings, etc.).
Initial Condition	User is browsing products on the catalog view
Procedure	1. User is on the catalog page. 2. User can see a list of products 3. User clicks on a specific product from the list.
Expected Outcome	All details of the product will be displayed to the user about the item he clicked including Car brand, year, price, milage, ratings, reviews ..etc.

Test ID	6
Category	Catalog
Requirements Coverage	UC M3: add a review for an item
Initial Condition	User is browsing products on the detailed view of the item
Procedure	1. Click on the add review button for the specific item in view. 2. User types the review in an input box. 3. User inputs a rating score. 4. User clicks submit review button.
Expected Outcome	The review that the user submitted will show on the detailed view page of the item along with other reviews from other users. The rating of the item will be adjusted based on the rating score given

Test ID	7
Category	Catalog
Requirements Coverage	UC M5: add an individual item to shopping cart
Initial Condition	User is browsing products on the detailed view of the item or on the catalog page
Procedure	1. Click on add to cart button associated with the product in the list or on the detailed page.
Expected Outcome	If the item is in stock, it will be added to the users shopping cart and the balance of the cart will be updated

Test ID	8
Category	Shopping Cart
Requirements Coverage	UC C1: view all items in the shopping cart and their information (price, etc.).
Initial Condition	User has an item in the cart
Procedure	1. User clicks on view shopping cart.
Expected Outcome	The shopping cart will list the items in it for the user to view, the full details of each item.

Test ID	9
Category	Shopping Cart
Requirements Coverage	UC C2: remove individual items from the shopping cart or increase/decrease the quantity. While doing so, the total bill should be updated.
Initial Condition	Users is on the shopping cart view and has some items in it
Procedure	1. User removes an item from the list or change the quantity of an item.
Expected Outcome	If the item is removed or the quantity is set to 0 then the item will not show to the user in the shopping cart view. Then the total price will be adjusted.

Test ID	10
Category	Shopping Cart
Requirements Coverage	UC C3: “Checkout” submit button indicating they wish to purchase the items in the shopping cart.
Initial Condition	Users is on the shopping cart view and has some items in it
Procedure	1. User clicks on the checkout button.
Expected Outcome	User redirected to the checkout view to either log into their account with a password, or create a new account, or checkout as a guest.

Test ID	11
Category	Checkout Cart
Requirements Coverage	UC P2: for a new account they enter their account name, password, and default billing and shipping information. The new account is submitted to the Order Processing service
Initial Condition	Users selected to create an account from the checkout view
Procedure	1. User enters account name. 2. User enters password. 3. User enters billing and shipping information.
Expected Outcome	A new account is created for the user, and the order is processed and the user is redirected to the confirm page.

Test ID	12
Category	Checkout Cart
Requirements Coverage	UC P3: to submit their order, they verify their billing and shipping information, and enter in their credit card number.
Initial Condition	The user has either checked out as a guest, or new account or logged in.
Procedure	1. User verifies the information displayed about the billing and shipping info. 2. User enters password. 3. User enters billing and shipping information.
Expected Outcome	A new account is created for the user, and the order is processed and the user is redirected to the confirm page
Test ID	13
Category	Checkout Cart
Requirements Coverage	UC P4: “Confirm order” button
Initial Condition	The user is on the confirm order page
Procedure	1. User has checked the order details and is ready to pay. 2. User clicks on the confirm order button.
Expected Outcome	The order will be processed in a way that every 2 consecutive orders are approved and the third will be denies, if the order is approved, it will display “Order Successfully Completed.” If it is denied, you it will display “Credit Card Authorization Failed.”

Test ID	14
Category	Admin page
Requirements Coverage	UC A1: generate a report with the items sold each month
Initial Condition	The user has logged in and authenticated as admin
Procedure	1. Admin clicks on the admin settings. 2. Admin click on generate report for items sold.
Expected Outcome	A report will be generated detailing the items sold, quality and total revenue of the store

Test ID	15
Category	Admin page
Requirements Coverage	UC A2: provide reports on the website usage
Initial Condition	The user has logged in and authenticated as admin
Procedure	1. Admin clicks on the admin settings. 2. Admin click on generate report for app usage
Expected Outcome	A report will be generated showing the usage of the application with details on the ip addresses, days of visit, what the user did on the site

Test ID	16
Category	Chatbot
Requirements Coverage	Ask questions to a chatbot able to answer basic questions and help customers navigate to a specific vehicle or support page
Initial Condition	The system is initialized
Procedure	1. User is on the stores main page. 2. User clicks on chat with bot button. 3. User inputs a question for the bot.
Expected Outcome	The bot will provide an answer to the users question, if user asked about a car then the bot will provide a link for the user to navigate that car.

Server-Side Implementation:

Runtime Environnement:

The backend is running on the NodeJS runtime which allows the execution of JavaScript code. The development build is written in TypeScript which is a superset of JavaScript that includes types for variables and objects.

Framework:

ExpressJS is used to handle HTTPS requests such as GET, POST, PUT and DELETE. This framework is lightweight and opinionated meaning that the developer has the freedom to structure the code and add their preferred libraries. The system follows a REST Architectural style by listening to a port and waiting for a request, once a request is received it is then routed and the controller handles it, then sends an HTTPS response with a status code and the microservices are able to perform CRUD operations this way.

Data Persistent

MySQL DB is used to persist the data for each microservice, and Prisma ORM is used to interact with the database. Prisma allows creating, querying, grouping by and updating the database with a higher level of abstraction using the model schema.

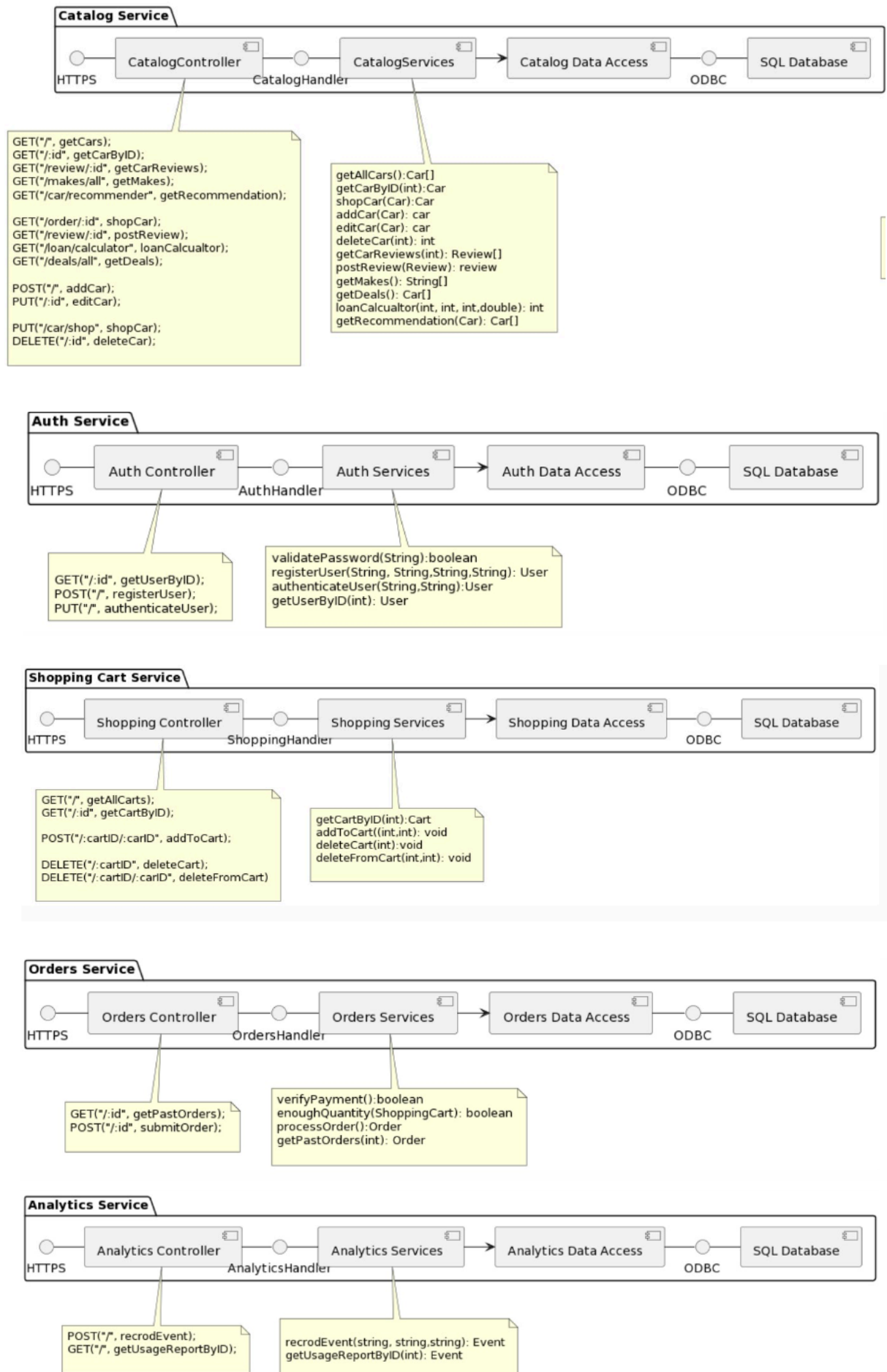
Hashing

ByCrypt is used to hash and salt the users passwords, where the hashed password is stored in the database.

Authentication

JWT web token library is used to generate tokens and sign the users token to verify their identity and authorise them to access resources

Detailed Design Diagram:



Activity Plan

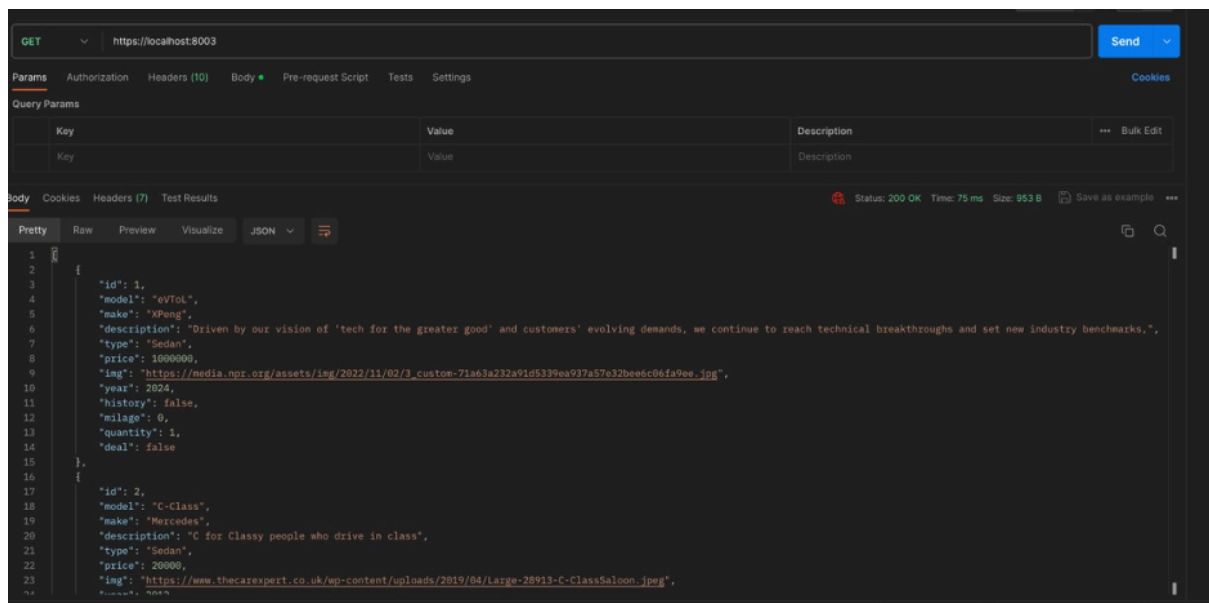
Activity	Participants	Date	Length	Summery
Kick off	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	October 22 nd	1 hour	Discussed the approach and the technologies to be used to do deliverable 2
Coding #1	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	October 26 th	2.5 hours	Meeting to start coding and assign work per student, each had to start with a microservice and set up the computers to work on.
Coding #2	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	November 1 st	2.5 hours	Meeting to catch up on progress and code the backend.
Coding #3	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	November 7 th	2.5 hours	Meeting to catch up on progress and code the backend again.

Peer review	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	November 10 th	1 hour	Meeting to review each others work and give advice for code reliability
Coding #4	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	November 13 th	2 hours	Discussed the front end and potential ways to do it and coded the backend.
Coding #5	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	November 17 th	1 hour	Finalized most of the code and started working on Deliverable 2 Documentation.
Hand in	Ali Adam Waleed Samouh Eyad Mahmoud Malik Samouh	November 18 th	1 hour	Agreed on final submission details and submitted.

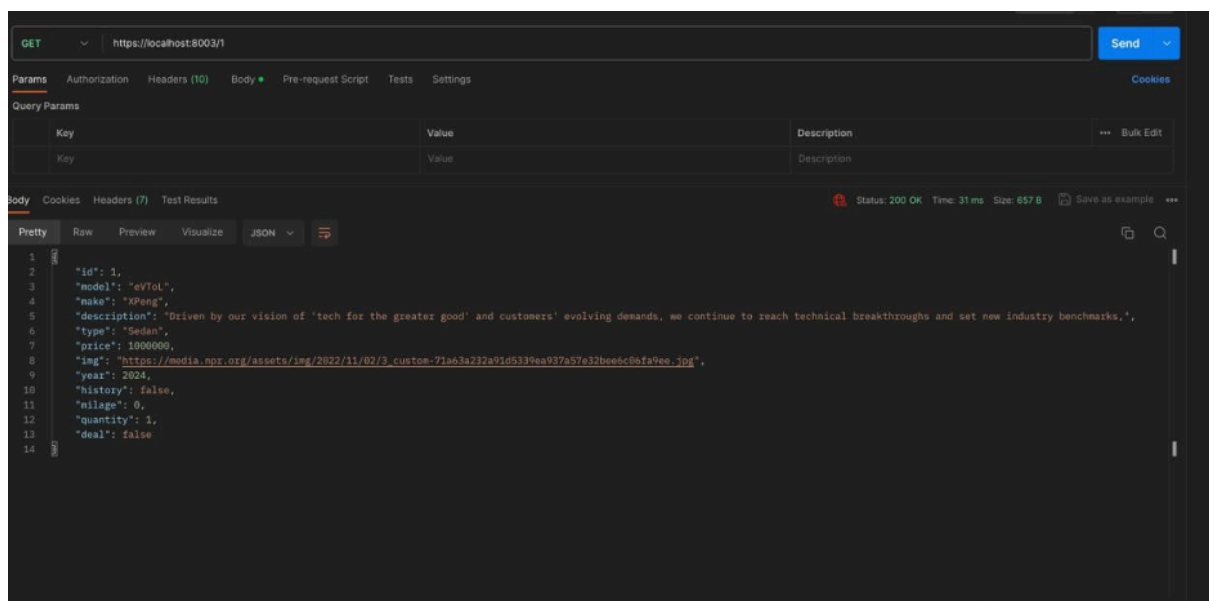
Postman Test:

- **Catalog Service:**

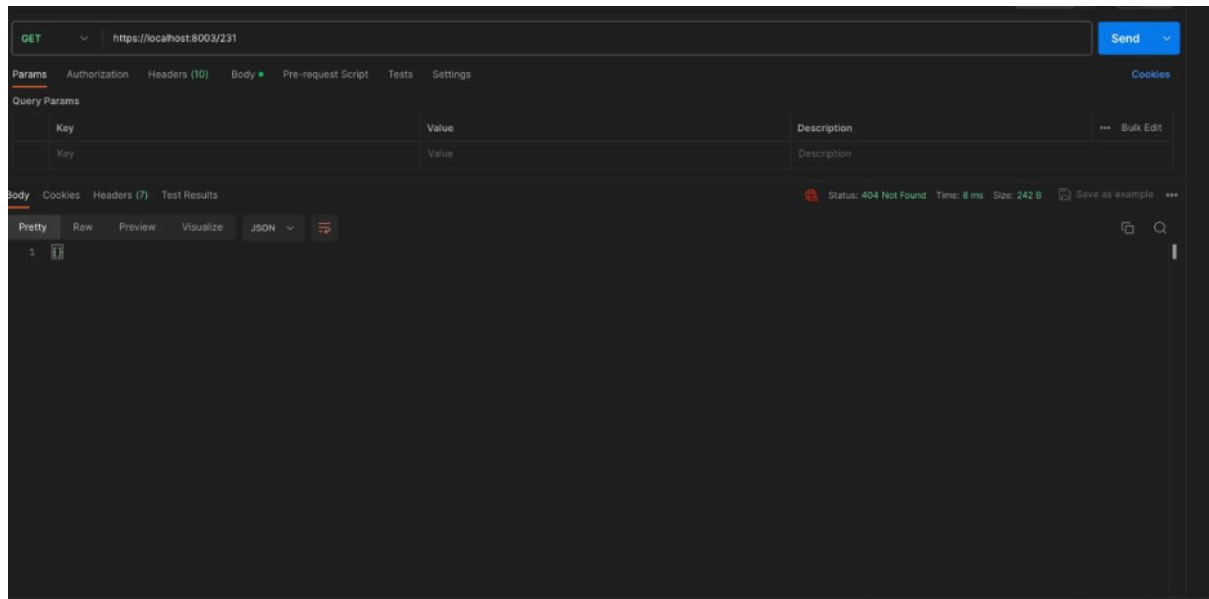
Get all products : GET("/"): Returns an array of all car objects



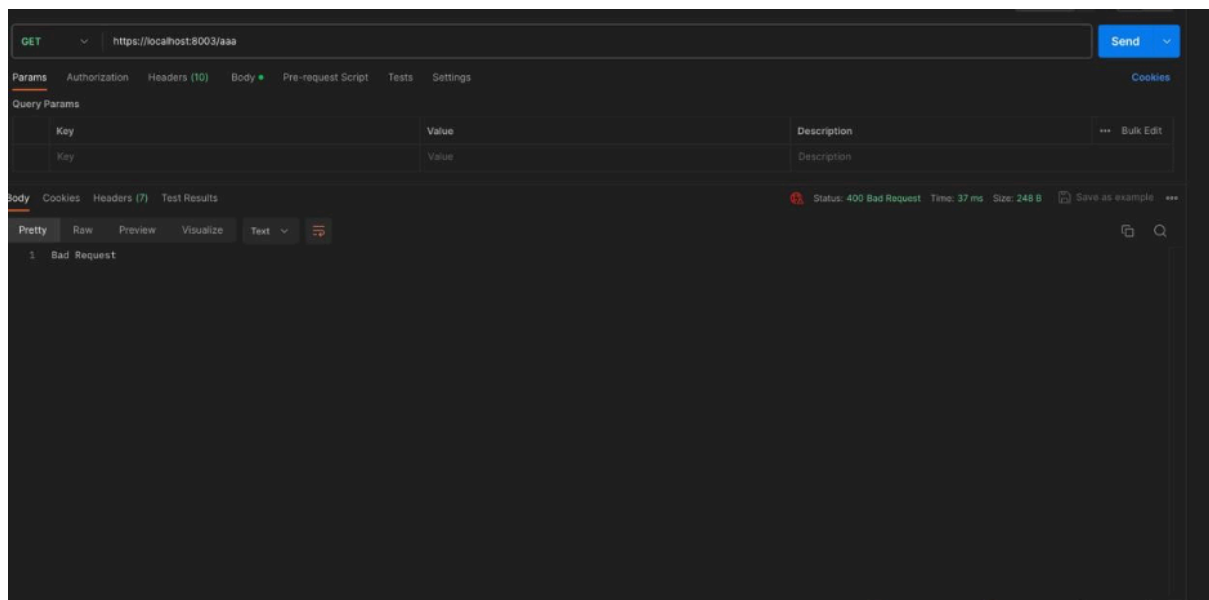
Get product by ID : GET("/:id"): Returns the car object



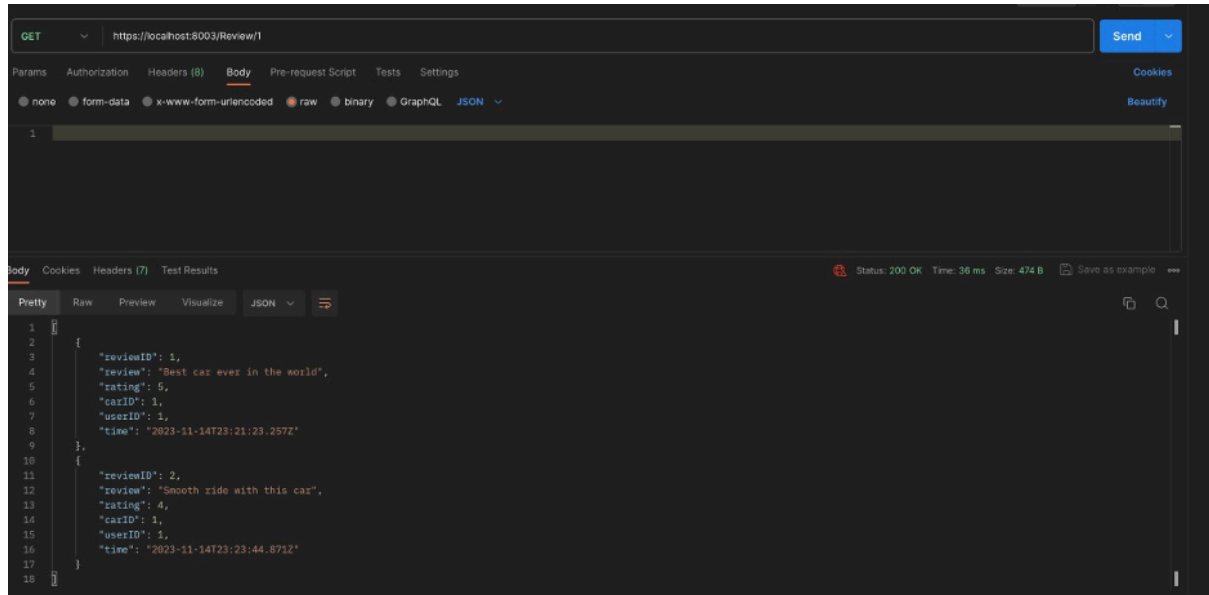
Get a car by ID that does not exist: return an empty object and 404 status code



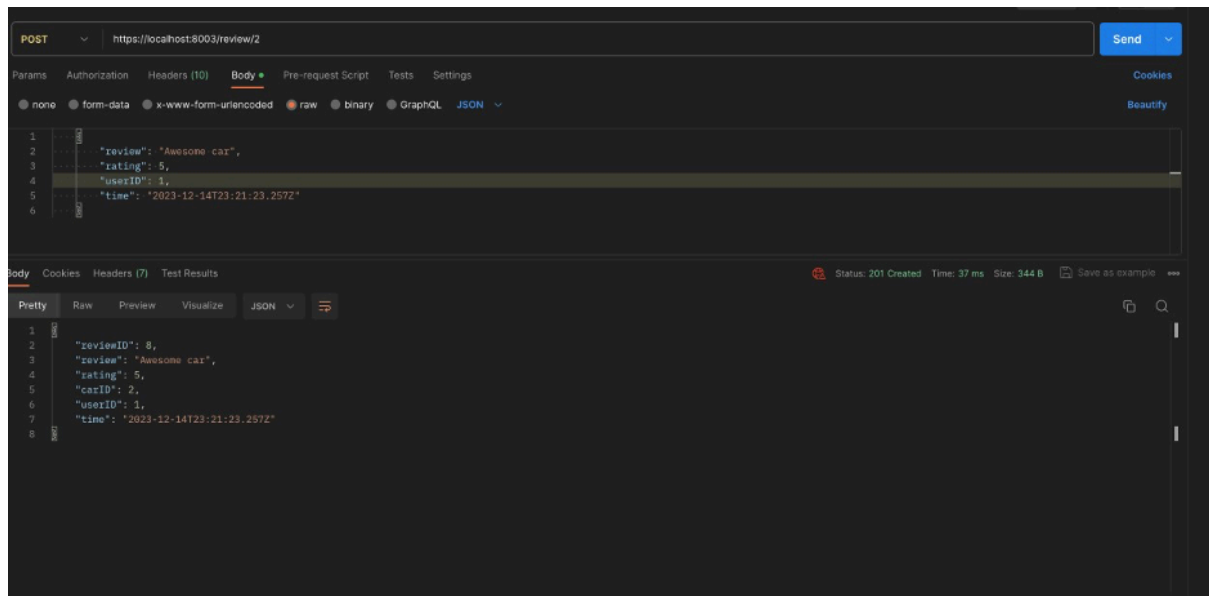
Input a string for id instead of a number: return “Bad request” and 400 status code



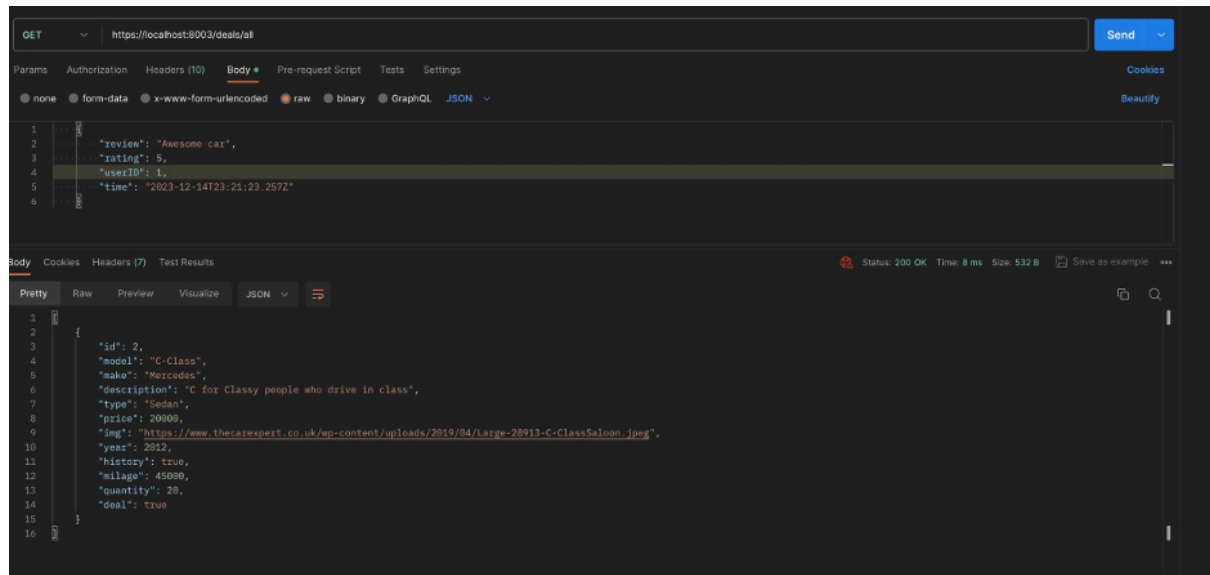
Get car reviews: GET(“/review/carID”): returns the array of review objects



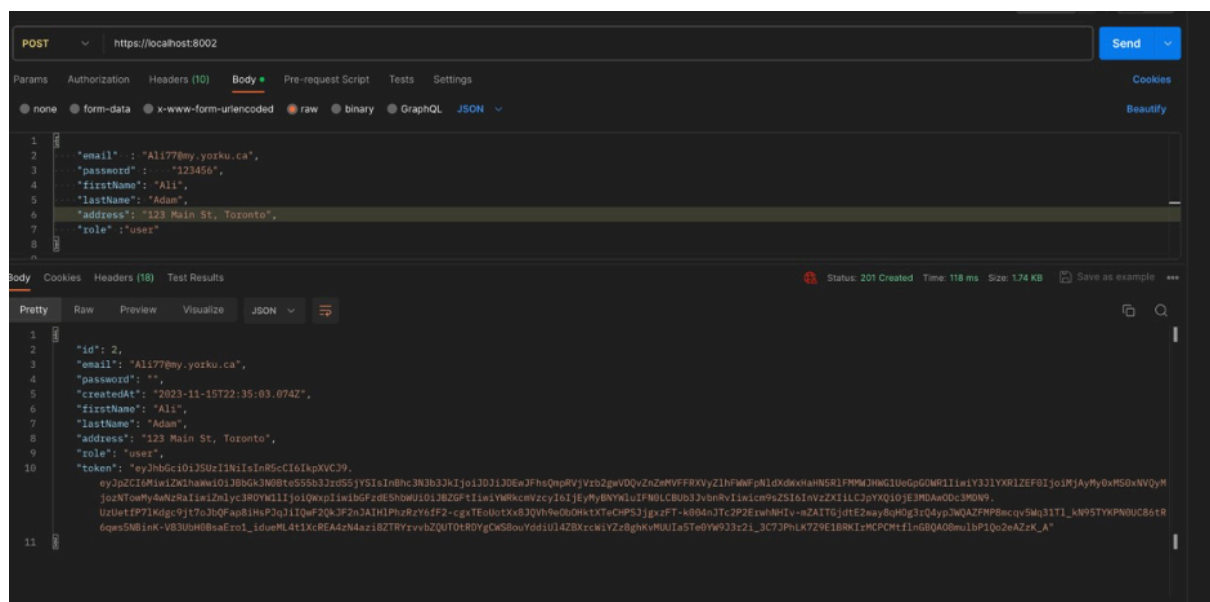
Post a car review : POST(“/review/carID”)



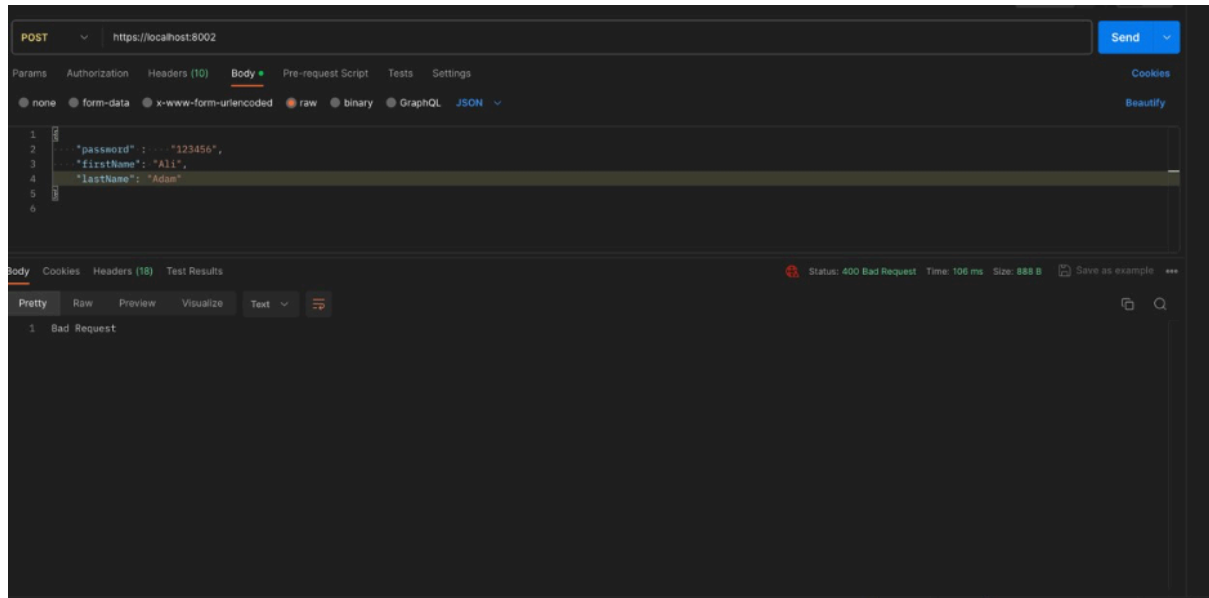
Get Deals: GET("/deals/all"):



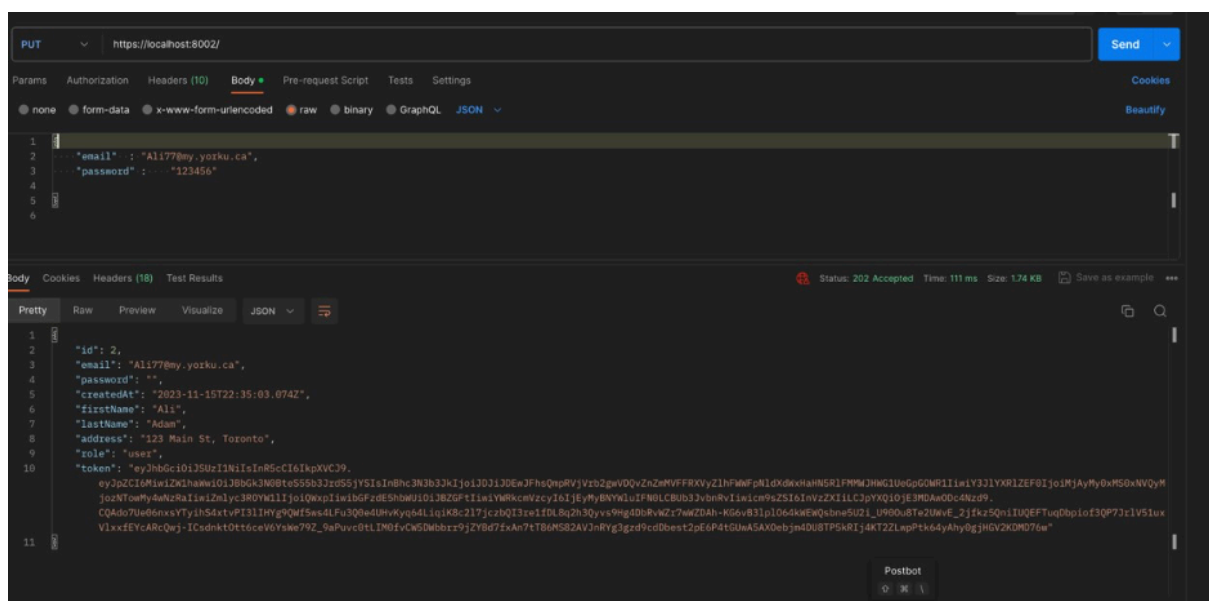
Register a user POST("/") : Returns the user with a token



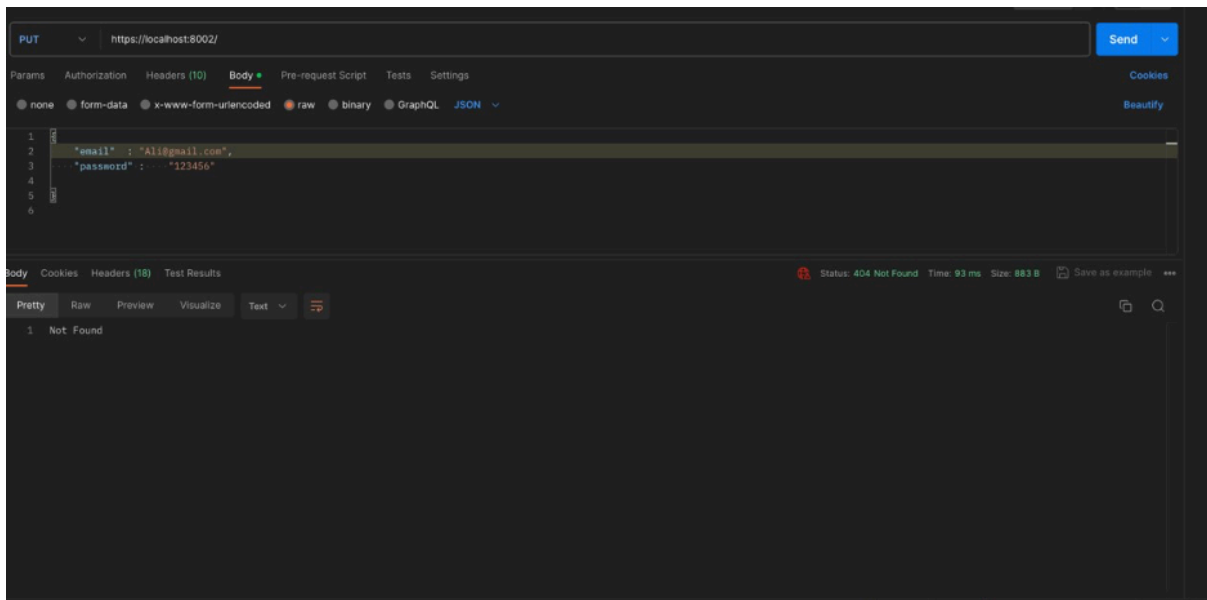
Register a user with missing arguments: Returns “Bad Request” with status 400



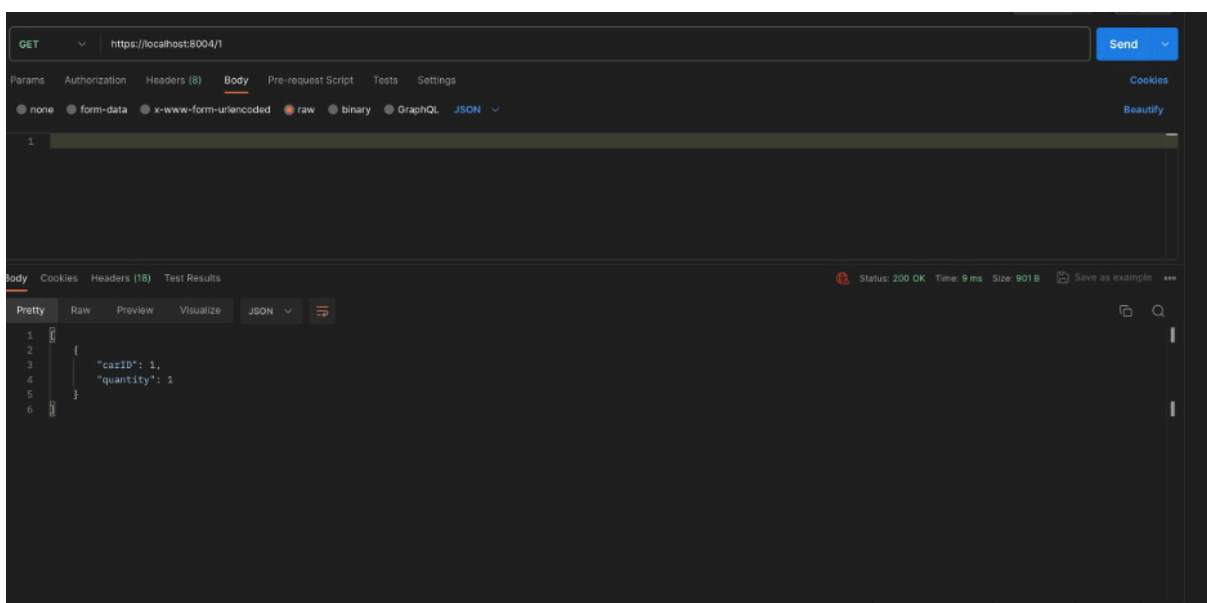
Login with email and password PUT("/"): Returns the user info with a token:



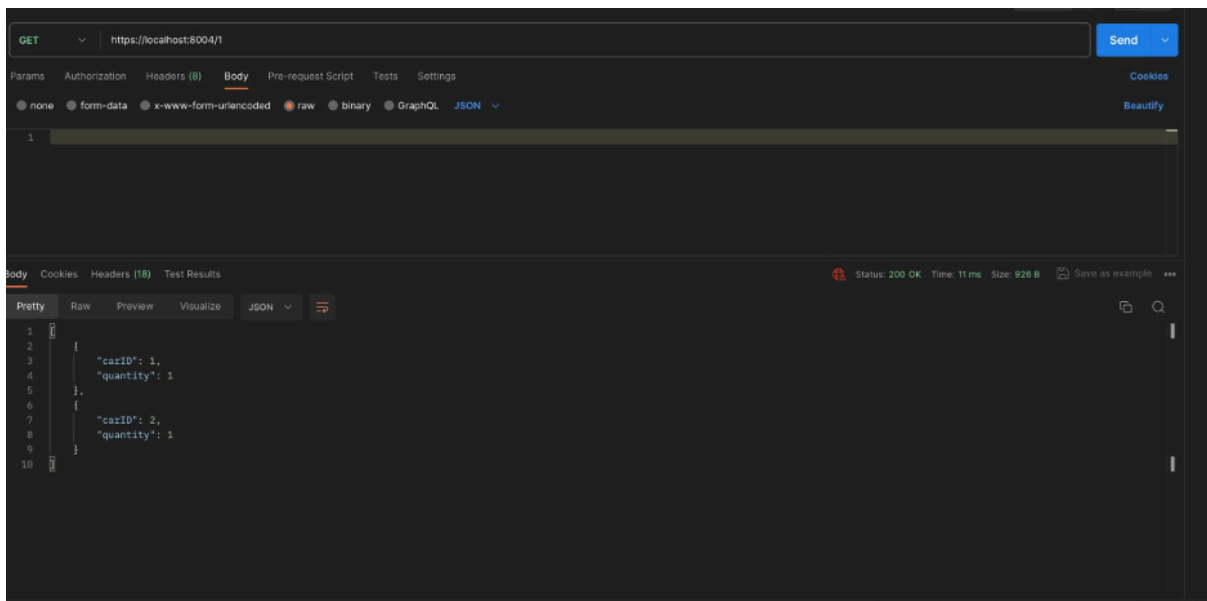
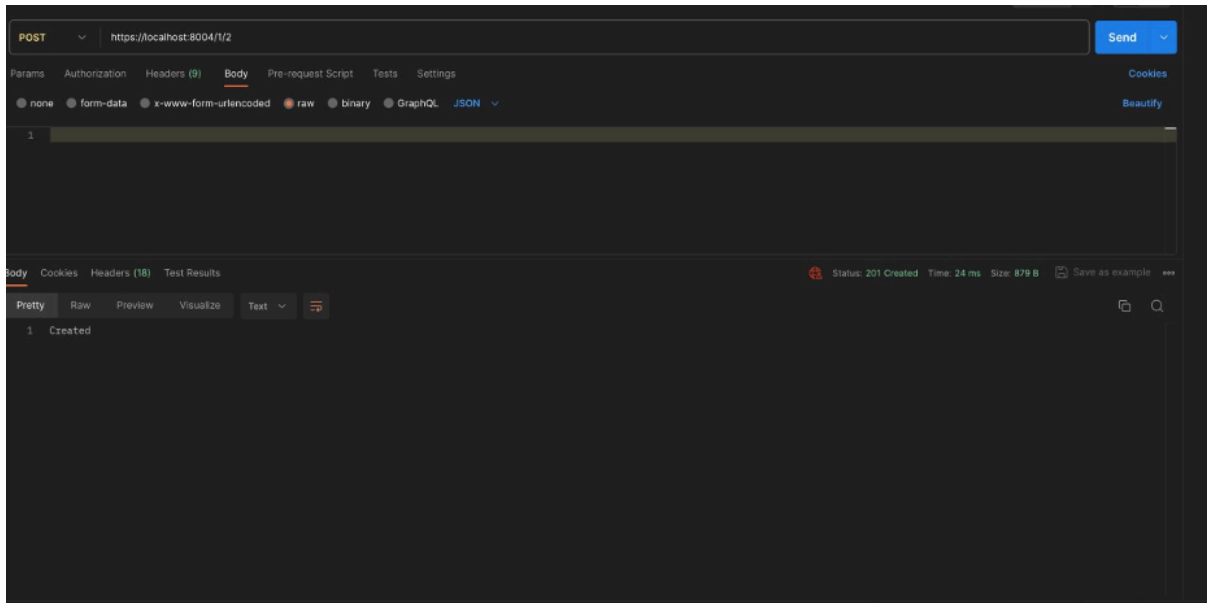
Login with a wrong email or password: Return 404 not found



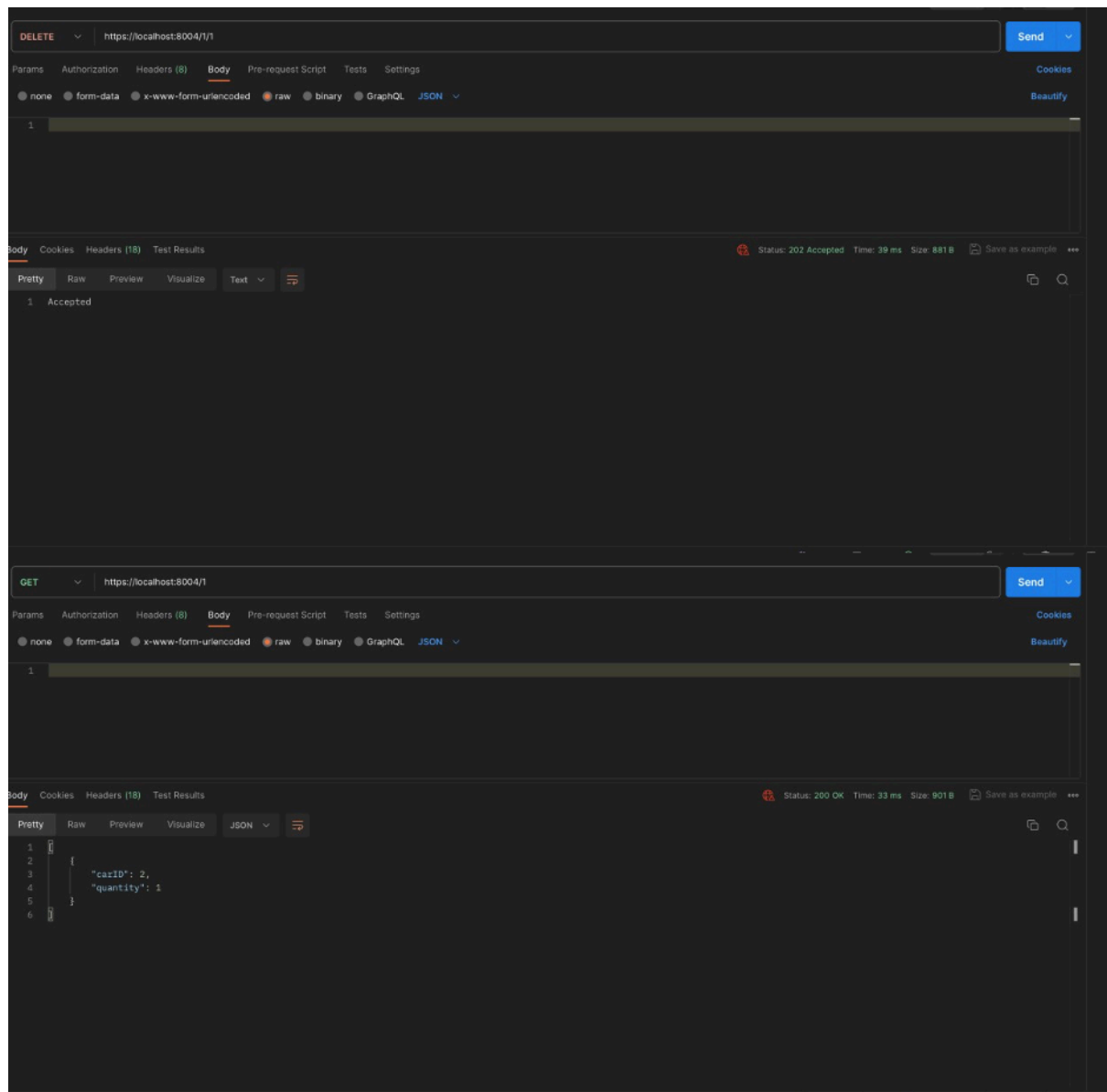
Get shopping cart by ID: GET("/:id")



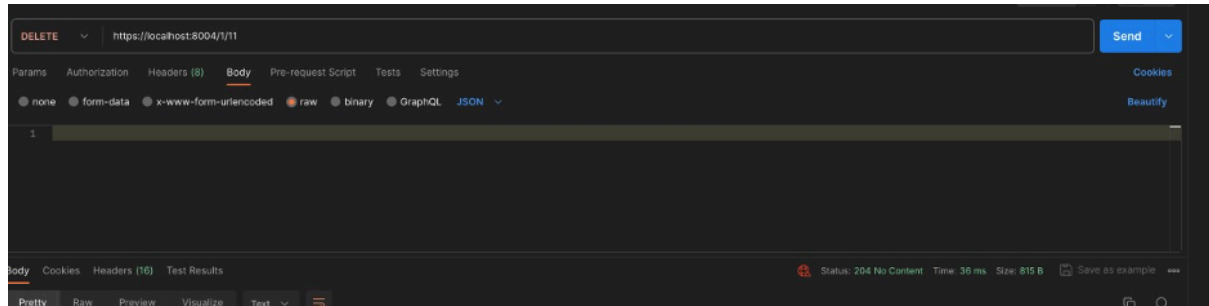
Add to shopping cart POST("/:cartID/:carID")



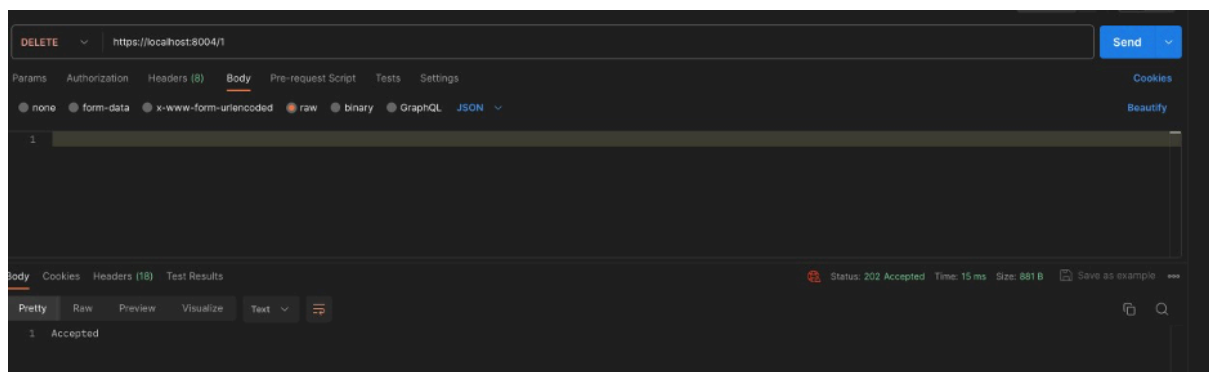
Delete a car from cart: DELETE("/cartID/carID"):



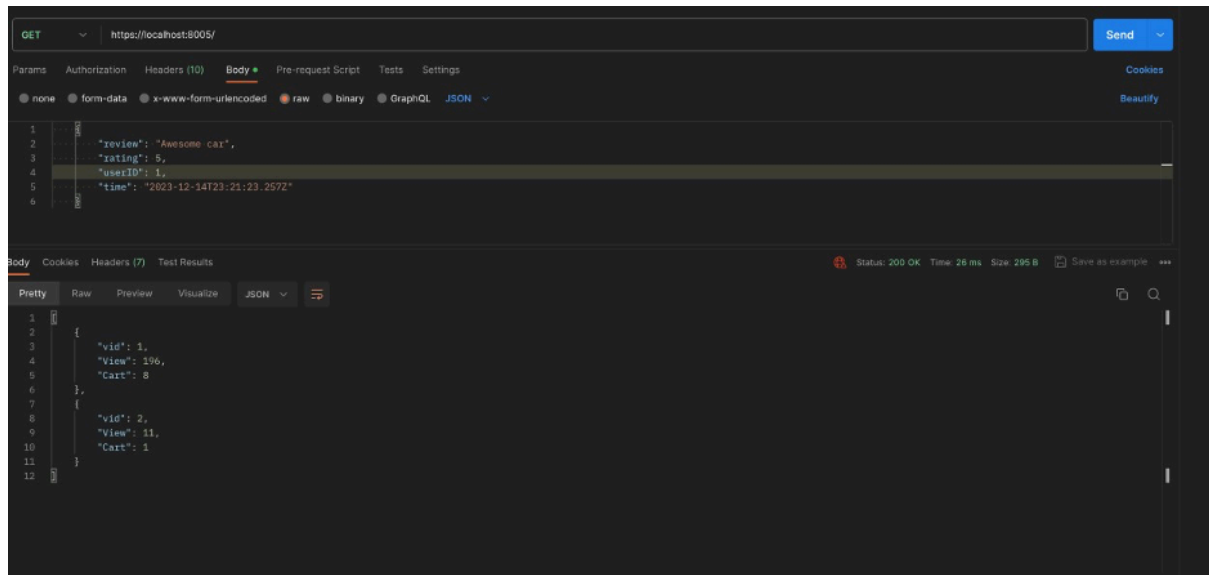
Deleting a car that does not exist in cart: Return status 204 no content;



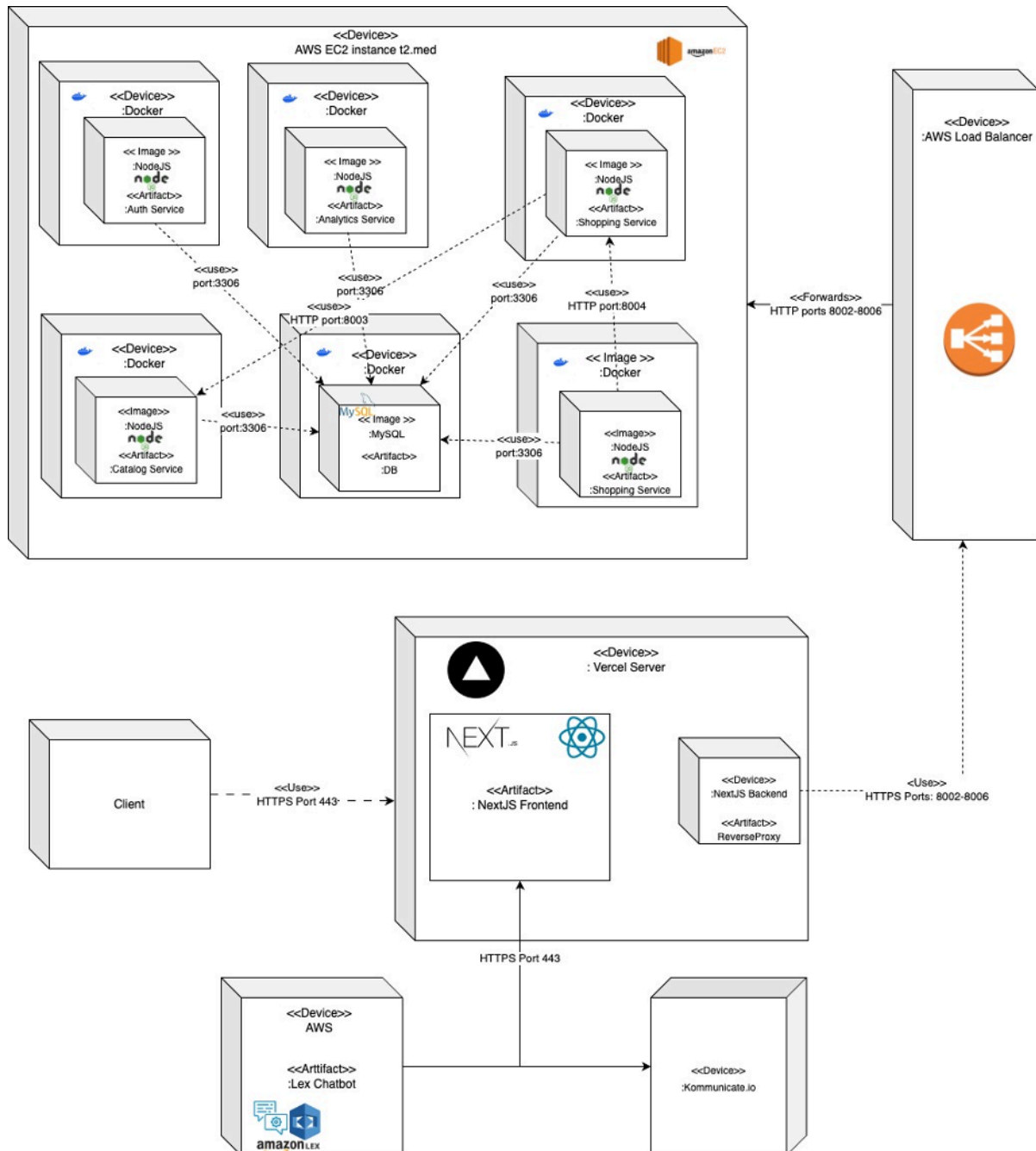
Delete the entire cart: DELETE("/cartID")



Get usage: return an array that contains the number of views, added to cart and orders for each car



Deployment:



Deployment Technologies:

Backend:

- Using docker, each microservice was made into a docker image built on a NodeJS image, this creates the environment and dependencies of each service. Then the images were uploaded into docker hub
- On AWS, an EC2 instance was created to download the docker images and containerize them along with a MySQL container
- Using Docker Compose, the containers were created and ports 8002-8006 were exposed from the EC2 machine, docker compose also creates an internal network for the containers to communicate
- An AWS Load balancer was set up with Port listener on 8002-8006, which forwards the HTTP requests to the microservice
-

Frontend:

- The NextJS system was deployed on Vercel, and an SSL Certificate was generated to make the connection over HTTPS
- Using the DNS resolver, the domain ecars.sale now points to the Vercel server
- The client can now use the frontend system of our application through the browser and interact with the AWS through the NextJS reverse proxy.

Quality Attributes

A)

Security:

- Used HTTPS to secure communications from the client to the front-end hosted on Vercel. Then Vercel communicates that request to the load balancer where the back-end is hosted.
- Vercel serves as a reverse proxy which does header destruction on the http request by decapsulating the request to the application layer then re-encapsulating it with its own generated header which is sent to the load balancer. This assures the load balancer gets the requests on the correct ports and does not allow for by-passing, and everything is following the intended routes. This also takes out the source IP from the packets being sent from the backend when being sent out, which means any attackers cant find out the IP address of the backend and cannot attempt to DDoS it.
- Website has an R3 "Certificate Authorities" Trusted SSL/TLS certificate, which is signed with the public key.
- A pair of keys is generated using the RS256 (RSA family) cryptographic algorithm to generate an asymmetric key to ensure safe TLS communications.
- Once a user is registered, the authentication microservices generates a token that is stored on the client's side to validate if the user requesting the resource is authorised to read/modify it
- The password entered on registration gets sent to the authentication microservice. It then goes through a one way hash using the blowfish hashing algorithm which is specialised in password hashing. The password goes through 10 rounds of salting (the algorithm runs $2^{10} = 1024$ times) then the ciphertext password gets stored into the database and the plaintext password gets deleted and never stored anywhere. This ensures that passwords remain a secret even if the database gets hacked.
- On sign in the password entered gets hashed and compared to the cipher text version in the database. Plain text password is also destroyed here and never stored.

Performance:

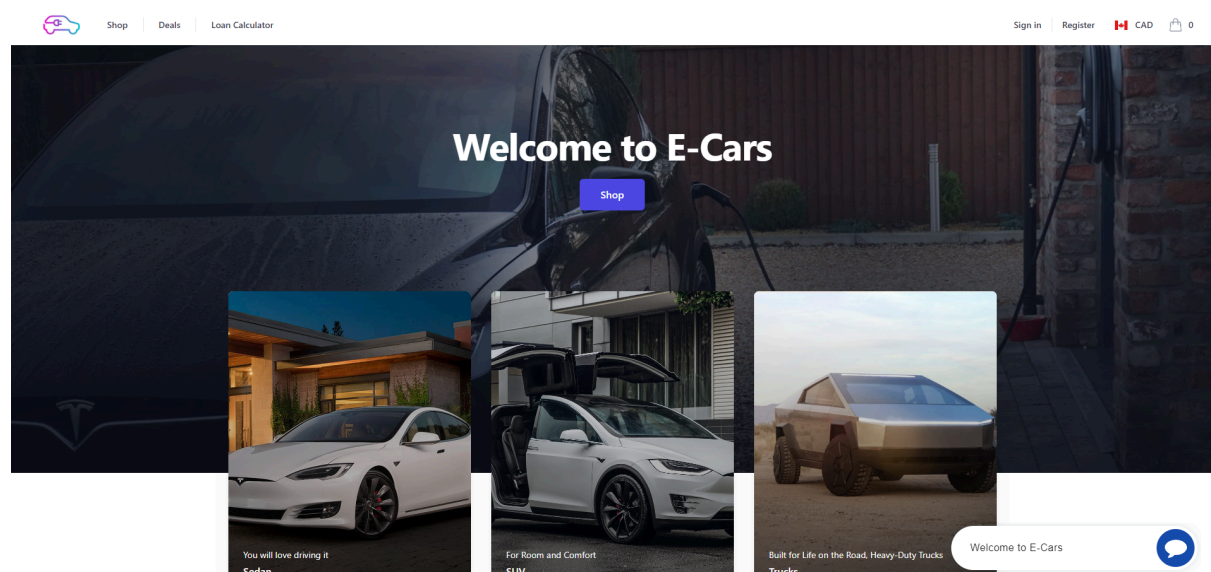
- Security measures added are efficient and don't risk much performance with redundancy. System has a good balance between security and performance.

- Using Node.js, built using the V8 Engine, allows the system to be efficient and quick because it deploys measures like non-blocking Asynchronous I/O. this means we can initiate many requests while waiting on responses to ensure seamless I/O
- System was built with the “Microservices” architecture, this allows for separation of concerns and load balancing. If a module has high load the system won't crash anything other than that one microservice.
- Hosting the front end on Vercel and using it as a proxy for requests allows for caching pages and responses, resulting in improved performance and responsiveness. This also decreases the load that gets directed to the load balancer. Which makes it more cost effective and of lower latency.

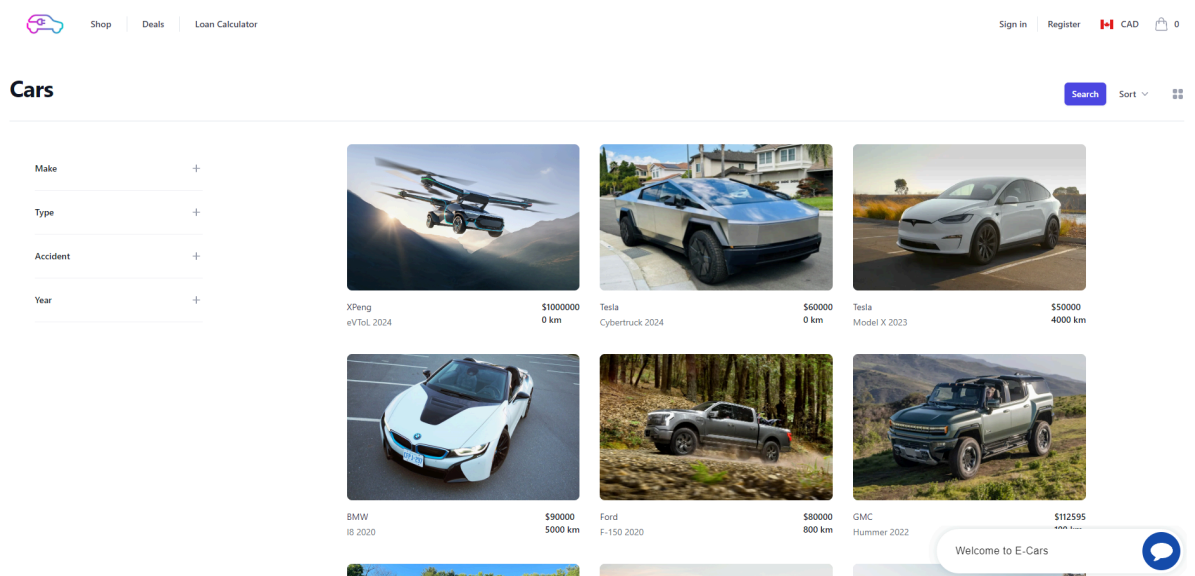
Scalability:

- The system can scale up by allocating more resources to the host machine on AWS EC2, for the moment we are using the t2.medium with 4GB of Ram, AWS can easily scale up the hardware based on the usage.
- The system is composed of microservices, which can scale out horizontally, by deploying multiple containers of the same service, this is useful since not all services scale in the same proportion, giving us the ability to have 2 catalogue services for example while keeping 1 instance of the others.
- With AWS Load Balancer, the incoming requests can be directed to different clusters based on several factors such as: the server with the least traffic, or the server that would execute the request in the least time, or based on the region.
- System is currently only hosted on US-East, and can be hosted on many more regions and zones around the world meaning it can scale up to a wide spread of geographic breadth by leveraging the AWS global infrastructure.

B) UI/UX



The main page used a dark and light colour scheme to increase contrast and make the options pop out more distinctly. The images of world leading electrical cars in the market are attached to not only grasp attention but also to give the user a better understanding of what we offer and feel welcomed and in the right place to purchase their next new electric car.



Our UI's main theme was simplicity, sleekness and futuristic. Our main intention was to make it clear to the user what each option does, that was achieved by using a single verb to describe each option. We also worked on decluttering the screen for the user by adding negative spacing between each option as seen between each car image and the one adjacent to it. A white background and black text theme was followed, this is to ensure maximum contrast between each segment and option, this also makes the images pop out and be more vibrant due to the simple background.

Hot Deals:



GMC Hummer 2022
\$112595



Mercedes EQS 2022
\$105550



Cadillac Lyriq 2024
\$60000

We can also observe the feedback on the screen as seen when we hover over the “GMC Hummer 2022” the window gets a light white screen with high opacity over it. This makes the system feel more interactive and tactile for the end user. All of these were done to ensure the highest quality of level user experience is achieved.

c)

Activity Plan:

Activity	Members attending	Date	Length	Summary
Kick-off for deliverable 3	All	November 18th	1hr	discussed the final phase of this project, the front end.
Coding assignments and work load discussions	All	November 21st	2hrs	Decided on who does what and when.
work session and code peer review #1	All	November 24th	1hr	Gathered and sat down to code and document.
work session and code peer review #2	All	November 27th	1hr	Gathered and sat down to code and document.
work session and code peer review #3	All	November 30th	1hr	Gathered and sat down to code and document.
practiced for the demo and rehearsals	Ali, Eyad	December 3rd	3hrs	Practiced the demo and presentation and flow of showcasing the work.
Demo day	All	December 4th	11am-2pm	Presented.
Hand in day + documentation wrap up	All	December 6th	4hrs	finalized documentation and collaborated on submitting the work.

D)

Frontend Implementation:

Framework:

The Front-end is implemented using the NextJS framework. It is built on React with support for Server-Side rendering (SSR).

NextJS uses the BFF pattern (Backend for frontend). Which means that there is a backend server that interacts with the client and serves them pages or can handle API routes. This backend server is not to be confused with the actual backend that is responsible for the services and interacting with the database.

The NextJS backend can render the requested page on the server, then serves a pre rendered page using HTML to the client, it is also capable of serving client-side components by sending the JS file to the client. The server-side components are useful for static pages or pages that don't require interactivity. These pages can also be cached and statically generated on the server.

The client-side components are used when there is some form of interaction between user and the website, where we would like some sections in the website to render dynamically based on events.

For example:

- The main landing page is a server-side component since it will be static and does not change.
- The navbar is a client-side component, upon logging in or registering, we would like to update the navbar dynamically, this can be done with client-side rendering, without having to reload the page and fetching the new html of the navbar
- The filtering page of cars is a client-side component, we can filter and search for cars without having to reload the page. Upon searching with new parameters, the new car data will come to the client and the page will dynamically render and update the component.

React:

The pages are written in React, which uses JSX, this allows us to write HTML with JavaScript. This way it is much easier to have the data dynamically rendered into the UI.

The most common React hooks we used were the `useState` and `useEffect`.

- `useEffect` is ran when the component is first mounted, this is useful if we would like to fetch data for the first time. `useEffect` can also have a dependency array that makes it execute whenever one of the elements in that array changes.
- `useState` creates a special type of variable that makes the page react to changes in that variable, for example if we use the `useState` variable in the page somewhere, and later on that variable has been updated, then the component would re-render and reflect the new changes.

Redux

Redux was used for the state management of the frontend. Using the redux store, we can store states that are accessible throughout the entire application, without having the need to pass the props from parent to child or even grandchild. This was used to store the user information and JWT token, as well as the cart since those two need to be accessed from many pages.

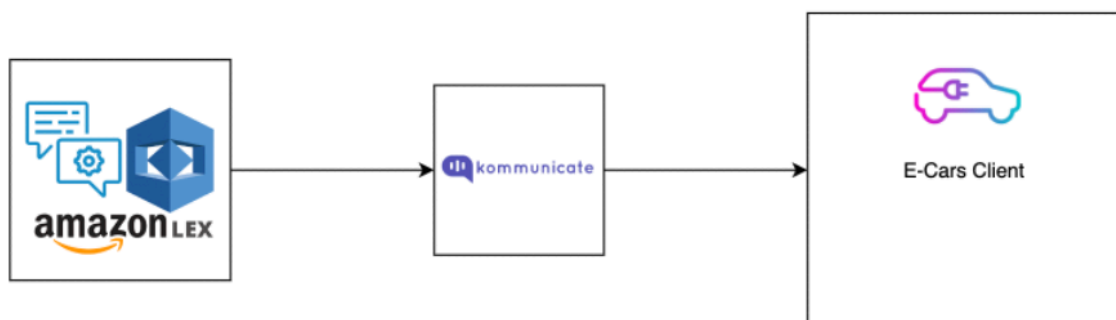
Redux-Persist

We used persistence for the redux state and stored the data in the local storage of the browser, this will allow us to keep the redux state even after the user reloads the page, or exists the browser, making the system stateful without the need to login every time

Tailwind CSS

Tailwind was used as the styling library for the application, allowing us to get the Ui up and running quickly using their components.

E) Chatbot



The chatbot was implemented using Amazon Lex, where it was trained based on prompts that would initiate a response, so far, the bot has been trained to respond to greetings such as Hello or Hi, if its asked for a car, it will reply by asking which brand,

then sends a link that would show the cars with that brand, It is also trained to send the link for the shopping cart and hot deals.

Kommunicate.io was used to link the amazon lex bot to our website, this library provides us with the Chat Ui and handles the interaction between the client and the amazon lex server.

F)

Demo

Feature to test/check during the demo (35% of Deliverables 3)	Mark	Comment made by the TA/prof
1) The system is cloud-native (4%)		
2) It is possible to register (2%)		
3) It is possible to sign in (2%)		
4) It is possible to sign out (2%)		
5) The system is secure (e.g., the system is using https in its url) (3%)		
6) It is possible to filter items (2%)		
7) It is possible to sort items (2%)		
8) It is possible to add items in the shopping cart (2%)		
9) It is possible to remove items from the shopping cart (2%)		
10) It is possible to check out items (2%)		
11) It is possible to write reviews on items and rate items using five stars (2%)		
12) It is possible to use the distinguished feature (2%)		
13) It is possible to use the chatbot (3%)		
14) It is possible to use the loan calculator (3%)		
15) It is possible to view hot deals (2%)		

G)

Strengths:

- Superb security and industry level policies running making this website of high quality.
- Cloud native which allows for high scalability and load balancing.
- Smooth and seamless UI allowing for maximum user experience.
- System ensures low latency and responsiveness, content is also generated dynamically similar to the cars to sort from, this get generated based on available car makes stored in the database

- Used the K nearest neighbour algorithm to find similar cars of the one the user is interested in.

Weaknesses:

- Lex AI chatbot limitations due to limited training
- K nearest neighbour algorithm runs on $O(k*m*\log(m))$ time complexity where k is the number of fields being compared to similarity and m is the number of items stored. This is a moderately efficient algorithm. It is not a problem when it comes to having a small data set but could potentially add latency to the system if the dataset was enormous.